

Lab Experiments 21-23

Data Handling & Visualization: Python Code & Output

Experiment 21

Point Estimation and Confidence Interval for Rare Element Concentration (NumPy)

PYTHON

Code

[illegible]

```

if sample_size > population_size:
    raise ValueError("Sample size cannot exceed the number of available measurements.")

# -----
# 3. Draw a random sample of the requested size
# -----
rng = np.random.default_rng()
sample = rng.choice(data, size=sample_size, replace=False)

# -----
# 4. Point estimation of the population mean
# -----
sample_mean = np.mean(sample)
sample_std = np.std(sample, ddof=1) # sample standard deviation

print("\n--- Point Estimation ---")
print(f"Sample size (n): {sample_size}")
print(f"Sample mean (point estimate of population mean): {sample_mean:.4f}")
print(f"Sample standard deviation: {sample_std:.4f}")

# -----
# 5. Confidence interval for the population mean (t-distribution,
#     since the population standard deviation is unknown)
# -----
alpha = 1 - confidence_level
t_critical = stats.t.ppf(1 - alpha / 2, df=sample_size - 1)
standard_error = sample_std / np.sqrt(sample_size)
margin_of_error = t_critical * standard_error

ci_lower = sample_mean - margin_of_error
ci_upper = sample_mean + margin_of_error

print("\n--- Confidence Interval ---")
print(f"Confidence level: {confidence_level * 100:.1f}%")
print(f"t-critical value: {t_critical:.4f}")
print(f"Margin of error: {margin_of_error:.4f}")
print(f"{confidence_level * 100:.1f}% Confidence Interval for the population mean: "
      f"({ci_lower:.4f}, {ci_upper:.4f})")

# -----
# 6. Minimum sample size needed to achieve the desired precision
#     n = (z * sigma / E)^2 -- using the sample std as an estimate
#     of the population std, and the z critical value for the
#     requested confidence level.
# -----
z_critical = stats.norm.ppf(1 - alpha / 2)
required_n = (z_critical * sample_std / desired_precision) ** 2
required_n = int(np.ceil(required_n))

print("\n--- Required Sample Size for Desired Precision ---")
print(f"Desired margin of error: {desired_precision}")
print(f"z-critical value: {z_critical:.4f}")
print(f"Minimum sample size required to achieve this precision "
      f"at {confidence_level * 100:.1f}% confidence: {required_n}")

```

Output

Loaded 300 concentration measurements from 'rare_elements.csv'.

Enter the sample size: 30

Enter the confidence level (e.g. 0.95 for 95%): 0.95

Enter the desired level of precision (margin of error, e.g. 0.1): 0.15

--- Point Estimation ---

Sample size (n): 30

Sample mean (point estimate of population mean): 5.2208

Sample standard deviation: 1.0025

--- Confidence Interval ---

Confidence level: 95.0%
t-critical value: 2.0452
Margin of error: 0.3743
95.0% Confidence Interval for the population mean: (4.8464, 5.5951)

--- Required Sample Size for Desired Precision ---

Desired margin of error: 0.15
z-critical value: 1.9600
Minimum sample size required to achieve this precision at 95.0% confidence: 172

Experiment 22

Customer Review Analysis: Confidence Interval for Mean Rating (Pandas)

PYTHON

Code

```
"""
Q22. Customer Review Analysis: Confidence Interval for Mean Rating
-----
Scenario: As an analyst for a popular online shopping website, the
task is to analyze customer reviews and provide insights on the
average rating and customer satisfaction level for a specific product
category, using data from "customer_reviews.csv" (contains product
ratings).

This program:
1. Loads the review ratings using Pandas.
2. Calculates descriptive statistics (mean rating, std deviation).
3. Calculates the customer satisfaction level (percentage of
ratings that are 4 stars or higher).
4. Uses Pandas together with SciPy to calculate a confidence
interval to estimate the true population mean rating.
"""

import os
import numpy as np
import pandas as pd
from scipy import stats

CSV_FILE = "customer_reviews.csv"
CONFIDENCE_LEVEL = 0.95
SATISFACTION_THRESHOLD = 4 # ratings >= 4 are considered "satisfied"

# -----
# Create a sample customer_reviews.csv if it does not already exist
# (so the script can be run/tested end-to-end).
# Remove this block once you have your real data file.
# -----
if not os.path.exists(CSV_FILE):
    rng = np.random.default_rng(11)
    n = 150
    ratings = rng.choice([1, 2, 3, 4, 5], size=n,
                        p=[0.03, 0.07, 0.15, 0.35, 0.40])
    sample = pd.DataFrame({
        "Review ID": range(1, n + 1),
        "Product Category": rng.choice(
            ["Electronics", "Home & Kitchen", "Fashion"], size=n),
        "Rating": ratings
    })
    sample.to_csv(CSV_FILE, index=False)
    print(f"'{CSV_FILE}' not found – a sample file was generated for demo purposes.\n")

# -----
# 1. Load the data with Pandas
# -----
df = pd.read_csv(CSV_FILE)
print(f"Loaded {len(df)} customer reviews from '{CSV_FILE}'.")
print(df.head(), "\n")

ratings = df["Rating"]

# -----
# 2. Descriptive statistics
# -----
mean_rating = ratings.mean()
std_rating = ratings.std() # sample standard deviation
```

```

n = ratings.count()

print("--- Descriptive Statistics ---")
print(f"Number of reviews:      {n}")
print(f"Average rating:         {mean_rating:.3f}")
print(f"Standard deviation:       {std_rating:.3f}")

# -----
# 3. Customer satisfaction level
# -----
satisfied = (ratings >= SATISFACTION_THRESHOLD).sum()
satisfaction_pct = satisfied / n * 100

print(f"\n--- Customer Satisfaction ---")
print(f"Reviews rated {SATISFACTION_THRESHOLD} stars or higher: {satisfied} / {n}")
print(f"Customer satisfaction level: {satisfaction_pct:.2f}%")

# -----
# 4. Confidence interval for the true population mean rating
# (t-distribution, population std unknown)
# -----
standard_error = std_rating / np.sqrt(n)
t_critical = stats.t.ppf(1 - (1 - CONFIDENCE_LEVEL) / 2, df=n - 1)
margin_of_error = t_critical * standard_error

ci_lower = mean_rating - margin_of_error
ci_upper = mean_rating + margin_of_error

print(f"\n--- {CONFIDENCE_LEVEL * 100:.0f}% Confidence Interval for Mean Rating ---")
print(f"Standard error: {standard_error:.4f}")
print(f"t-critical:      {t_critical:.4f}")
print(f"Margin of error: {margin_of_error:.4f}")
print(f"{CONFIDENCE_LEVEL * 100:.0f}% CI for the true population mean rating: "
      f"({ci_lower:.3f}, {ci_upper:.3f})")

# Cross-check using SciPy's built-in interval function
ci_scipy = stats.t.interval(CONFIDENCE_LEVEL, df=n - 1,
                           loc=mean_rating, scale=standard_error)
print(f"(SciPy stats.t.interval cross-check: "
      f"({ci_scipy[0]:.3f}, {ci_scipy[1]:.3f}))")

```

Output

Loaded 150 customer reviews from 'customer_reviews.csv'.

	Review ID	Product Category	Rating
0	1	Electronics	3
1	2	Electronics	4
2	3	Electronics	5
3	4	Home & Kitchen	1
4	5	Electronics	3

--- Descriptive Statistics ---

Number of reviews: 150
Average rating: 3.840
Standard deviation: 1.142

--- Customer Satisfaction ---

Reviews rated 4 stars or higher: 98 / 150
Customer satisfaction level: 65.33%

--- 95% Confidence Interval for Mean Rating ---

Standard error: 0.0932
t-critical: 1.9760
Margin of error: 0.1842
95% CI for the true population mean rating: (3.656, 4.024)
(SciPy stats.t.interval cross-check: (3.656, 4.024))

Experiment 23

Clinical Trial Hypothesis Testing: Placebo vs New Drug (Matplotlib)

PYTHON

Code

```
"""
Q23. Clinical Trial Hypothesis Testing: Placebo vs New Drug
-----
Scenario: A researcher in a medical lab is investigating the
effectiveness of a new treatment. Data was collected from a clinical
trial with two groups: a control group receiving a placebo, and a
treatment group receiving the new drug. The goal is to analyze the
data using hypothesis testing, calculate the p-value to determine if
the new treatment has a statistically significant effect compared to
the placebo, and visualize the data and the p-value using Matplotlib.

Hypotheses:
    H0 (null):      There is no difference in the mean outcome
                    between the placebo group and the treatment group.
    H1 (alternative): The treatment group's mean outcome is
                    significantly different from the placebo group's.

This program:
    1. Loads (or simulates) the control and treatment group data.
    2. Performs an independent two-sample t-test.
    3. Reports the t-statistic and p-value, and states the conclusion
       at a 5% significance level.
    4. Visualizes the two groups (boxplot comparison) and the p-value
       relative to the significance threshold.
"""

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

CSV_FILE = "clinical_trial.csv"
ALPHA = 0.05 # significance level

# -----
# Create a sample clinical_trial.csv if it does not already exist
# (so the script can be run/tested end-to-end).
# Remove this block once you have your real data file.
# Expected columns: 'Group' ('Control' / 'Treatment'), 'Outcome'
# (e.g. an improvement score / recovery measure).
# -----
if not os.path.exists(CSV_FILE):
    rng = np.random.default_rng(3)
    n_per_group = 40
    control = rng.normal(loc=50, scale=8, size=n_per_group)
    treatment = rng.normal(loc=56, scale=8, size=n_per_group)
    sample = pd.DataFrame({
        "Group": ["Control"] * n_per_group + ["Treatment"] * n_per_group,
        "Outcome": np.round(np.concatenate([control, treatment]), 2)
    })
    sample.to_csv(CSV_FILE, index=False)
    print(f"'{CSV_FILE}' not found – a sample file was generated for demo purposes.\n")

# -----
# 1. Load the data
# -----
df = pd.read_csv(CSV_FILE)
control_group = df.loc[df["Group"] == "Control", "Outcome"]
treatment_group = df.loc[df["Group"] == "Treatment", "Outcome"]
```

```

print(f"Control group:  n={len(control_group)}, "
      f"mean={control_group.mean():.3f}, std={control_group.std():.3f}")
print(f"Treatment group: n={len(treatment_group)}, "
      f"mean={treatment_group.mean():.3f}, std={treatment_group.std():.3f}\n")

# -----
# 2. Independent two-sample t-test (Welch's t-test: unequal variances)
# -----
t_stat, p_value = stats.ttest_ind(treatment_group, control_group, equal_var=False)

print("--- Hypothesis Test: Independent Two-Sample t-test ---")
print(f"H0: No difference between control and treatment group means")
print(f"H1: Treatment group mean differs from control group mean")
print(f"t-statistic: {t_stat:.4f}")
print(f"p-value:      {p_value:.6f}")

# -----
# 3. Conclusion at the chosen significance level
# -----
if p_value < ALPHA:
    conclusion = (f"p-value ({p_value:.6f}) < alpha ({ALPHA}) -> Reject H0.\n"
                  f"The new treatment has a statistically significant effect "
                  f"compared to the placebo.")
else:
    conclusion = (f"p-value ({p_value:.6f}) >= alpha ({ALPHA}) -> Fail to reject H0.\n"
                  f"There is not enough evidence that the new treatment has a "
                  f"statistically significant effect compared to the placebo.")
print(f"\nConclusion (alpha = {ALPHA}): \n{conclusion}\n")

# -----
# 4. Visualization: boxplot comparison + p-value bar
# -----
fig, axes = plt.subplots(1, 2, figsize=(11, 5))

# -- Boxplot comparing the two groups --
axes[0].boxplot([control_group, treatment_group],
                 tick_labels=["Control (Placebo)", "Treatment (New Drug)"],
                 patch_artist=True,
                 boxprops=dict(facecolor="lightblue"))
axes[0].set_title("Outcome Distribution: Control vs Treatment")
axes[0].set_ylabel("Outcome")

# -- p-value visualization against the significance threshold --
axes[1].bar(["p-value"], [p_value], color="darkorange", width=0.4)
axes[1].axhline(y=ALPHA, color="red", linestyle="--",
                 label=f"Significance level (alpha = {ALPHA})")
axes[1].set_ylim(0, max(p_value, ALPHA) * 1.5 + 0.01)
axes[1].set_title("p-value vs Significance Threshold")
axes[1].set_ylabel("p-value")
axes[1].text(0, p_value, f"{p_value:.4f}", ha="center", va="bottom")
axes[1].legend()

plt.tight_layout()
plt.savefig("clinical_trial_ttest.png", dpi=150)
plt.close()

print("Saved: clinical_trial_ttest.png")

```

Output

```

Control group:  n=40, mean=49.671, std=9.401
Treatment group: n=40, mean=54.807, std=8.173

--- Hypothesis Test: Independent Two-Sample t-test ---
H0: No difference between control and treatment group means
H1: Treatment group mean differs from control group mean
t-statistic: 2.6080
p-value:      0.010947
Conclusion (alpha = 0.05):

```

p-value (0.010947) < alpha (0.05) -> Reject H0.
The new treatment has a statistically significant effect compared to the placebo.

Saved: clinical_trial_ttest.png

Group Comparison and p-value Visualization

